HLR Exercise

In this exercise, we'll guide you through using the Goethe-HPC cluster for deep learning. Your job is to train a *Vision Transformer* on the *Fashion MNIST* dataset, experimenting with patch sizes and embedding dimensions. The idea is to use a single node with eight GPUs to train eight models simultaneously, each with unique parameters.

Remember: Nodes are used exclusively, i.e. only whole nodes are allocated for a job and no other job can use the same nodes concurrently.

Technical Details:

The dataset is implemented in the `torchvision` package using `torchvision.datasets.FashionMNIST`. We'll use the official training and testing split. Each model should be trained for at least 10 epochs with cross-entropy loss, and the accuracy on the test set, training time, and epoch time should be reported. Employ a batch size of 128, a learning rate of 1e-4, and an *Adam* optimizer. Feel free to use other parameters or augmentations.

The Vision Transformer is implemented in the `timm` library using `timm.models.VisionTransformer`. Keep the default configuration with 12 layers and 12 heads, adjusting the image size and the number of channels according to *Fashion MNIST*. To utilize one node efficiently, we aim to test a grid of 8 parameters.
Use patch sizes `[12, 7, 4, 2]` with embedding dimensions of `[384, 192]`.

Details:

1. Connect to the server using SSH:
   ```
   ssh <username>@goethe.hhlr-gu.de
   ```
2. Create a *conda* environment and install PyTorch for ROCm[1]
3. Create a script that contains all the training functionalities. It should be able to train a single model on a single GPU and should be tested on COLAB, to reduce the cluster usage. The main function expects a configuration of the parameters as well as a parameter that specifies the device.
4. Create a script that generates the eight config files i.e. as JSON files.
5. Create a script that is supposed to be called by SLURM as the main running script: `run.py`. When specifying n tasks, SLURM will run this script n times:
   1. Extract the process ID using `$SLURM_PROCID` (see `os.environ`)
   2. Open the configuration file that corresponds to the process ID

[1] PyTorch abstracts the ROCm support to the Cuda backend, which means you can specify your device in the same way as with NVIDIA GPUs

    3. Assign a unique GPU to the configuration
    4. Run the training script with the assigned configuration
6. Create a `jobscript.sh` containing the resource allocation:
    1. Specify all needed parameters i.e. CPU, Memory, Runtime. Look at the specification table to see the hardware of the GPU nodes.
    2. Set the number of tasks according to the number of GPUs
    3. Please set your mail in the `—mail-user` flag and adjust `—mail-type` accordingly
    4. Activate the correct Conda environment
    5. Run the running script `run.py` with `SRUN` command
7. Train the models using `sbatch jobscript.sh`. This should take approx. 30min for the slowest model.